Cohort SQL Assessment — Hotel Booking System

Participants first run the setup script below in PostgreSQL. Then they complete the **30 assignments without answers**.

1. Create Database

```
CREATE DATABASE hotel_assessment;
```

Connect to `hotel_assessment`, then create the following tables.

2. Create Tables

```
CREATE TABLE guests (
    guest_id INT PRIMARY KEY,
    guest_name VARCHAR(100),
    city VARCHAR(50),
    membership_level VARCHAR(20)
);


CREATE TABLE rooms (
    room_id INT PRIMARY KEY,
    room_type VARCHAR(50),
    price_per_night DECIMAL(10,2),
    floor_number INT,
    status VARCHAR(20)
);


CREATE TABLE bookings (
    booking_id INT PRIMARY KEY,
    guest_id INT,
    room_id INT,
    check_in_date DATE,
```

```
    check_out_date DATE,
    booking_status VARCHAR(30),

    FOREIGN KEY (guest_id)
        REFERENCES guests(guest_id),

    FOREIGN KEY (room_id)
        REFERENCES rooms(room_id)
);


CREATE TABLE services (
    service_id INT PRIMARY KEY,
    service_name VARCHAR(100),
    service_price DECIMAL(10,2)
);


CREATE TABLE booking_services (
    booking_service_id INT PRIMARY KEY,
    booking_id INT,
    service_id INT,
    quantity INT,

    FOREIGN KEY (booking_id)
        REFERENCES bookings(booking_id),

    FOREIGN KEY (service_id)
        REFERENCES services(service_id)
);
```

---

3. Insert Data

Guests

```
INSERT INTO guests VALUES
(1, 'Aditya Kulkarni', 'Pune', 'Gold'),
(2, 'Farah Siddiqui', 'Hyderabad', 'Silver'),
(3, 'Joel Mathew', 'Kochi', 'Gold'),
(4, 'Tanvi Deshmukh', 'Nagpur', 'Standard'),
(5, 'Ritesh Batra', 'Delhi', 'Silver'),
```

```
(6, 'Navya Krishnan', 'Chennai', 'Standard'),
(7, 'Imran Sheikh', 'Mumbai', 'Gold'),
(8, 'Keerthi Nair', 'Bengaluru', 'Standard'),
(9, 'Arvind Sethi', 'Jaipur', 'Silver'),
(10, 'Simran Gill', 'Chandigarh', 'Standard');
```

Notice that some guests will intentionally have **no bookings**.

Rooms

```
INSERT INTO rooms VALUES
(101, 'Standard', 3200, 1, 'Available'),
(102, 'Standard', 3500, 1, 'Occupied'),
(201, 'Deluxe', 5200, 2, 'Available'),
(202, 'Deluxe', 5500, 2, 'Occupied'),
(301, 'Suite', 8500, 3, 'Available'),
(302, 'Suite', 9000, 3, 'Maintenance'),
(401, 'Executive', 6800, 4, 'Available'),
(402, 'Executive', 7200, 4, 'Occupied');
```

Bookings

```
INSERT INTO bookings VALUES
(1001, 1, 202, '2026-10-01', '2026-10-04', 'Completed'),
(1002, 2, 102, '2026-10-03', '2026-10-06', 'Completed'),
(1003, 3, 301, '2026-10-05', '2026-10-08', 'Confirmed'),
(1004, 4, 201, '2026-10-07', '2026-10-09', 'Cancelled'),
(1005, 1, 401, '2026-10-10', '2026-10-13', 'Confirmed'),
(1006, 5, 101, '2026-10-11', '2026-10-14', 'Confirmed'),
(1007, 7, 402, '2026-10-12', '2026-10-16', 'Confirmed'),
(1008, 2, 201, '2026-10-18', '2026-10-20', 'Confirmed');
```

Guests `6`, `8`, `9`, and `10` have no bookings.

Services

```
INSERT INTO services VALUES
(501, 'Breakfast Buffet', 750),
(502, 'Airport Pickup', 1200),
(503, 'Laundry', 500),
```

```
(504, 'Spa Session', 1800),
(505, 'Dinner Buffet', 1100);
```

Booking Services

```
INSERT INTO booking_services VALUES
(1, 1001, 501, 3),
(2, 1001, 503, 1),
(3, 1002, 502, 1),
(4, 1002, 501, 2),
(5, 1003, 504, 2),
(6, 1003, 505, 2),
(7, 1005, 501, 3),
(8, 1005, 502, 1),
(9, 1006, 503, 2),
(10, 1007, 504, 1),
(11, 1007, 505, 3);
```

Assessment – 30 Assignments

Basic SQL

1. Display all records from the `guests` table.
2. Display only `guest_name`, `city`, and `membership_level`.
3. Display all rooms whose price per night is greater than ₹5,000.
4. Display all rooms whose status is `Available`.
5. Find all guests belonging to the `Gold` membership level.
6. Display bookings whose status is `Confirmed`.
7. Display bookings sorted by `check_in_date`.
8. Display rooms sorted from highest to lowest `price_per_night`.

Filtering and Calculations

9. Find rooms priced between ₹4,000 and ₹8,000.
10. Find guests from either `Hyderabad`, `Pune`, or `Mumbai`.
11. Count the total number of rooms.
12. Find the average room price.
13. Find the most expensive room price.
14. Find the cheapest room price.
15. Count how many rooms exist for each `room_type`.

GROUP BY / Aggregation

16. Find the average room price for each room type.
17. Count how many guests belong to each membership level.
18. Count the number of bookings for each booking status.
19. Find the total value of all services in the `services` table.
20. Find the average service price.

JOIN Assignments

21. Guest + Booking

Display:

```
booking_id
guest_name
city
check_in_date
```

```
check_out_date
booking_status
```

Use `guests` and `bookings`.

---

22. Booking + Room

Display:

```
booking_id
room_type
price_per_night
check_in_date
check_out_date
```

---

23. Guest + Booking + Room

Display:

```
guest_name
room_type
price_per_night
booking_status
```

Use three tables.

---

24. Booking + Service

Display:

```
booking_id
service_name
quantity
service_price
```

Use:

```
bookings
booking_services
services
```

---

25. Calculate Service Cost

For every service used in a booking, calculate:

```
service cost = quantity × service_price
```

Display:

```
booking_id
service_name
quantity
service_price
service_cost
```

---

26. Complete Booking Details

Join:

```
guests
bookings
rooms
booking_services
services
```

Display:

```
guest_name
booking_id
room_type
service_name
quantity
booking_status
```

LEFT JOIN Assignments

27. Find all guests and their bookings

Display every guest, including guests who have **never made a booking**.

Output:

```
guest_name
booking_id
booking_status
```

Use a `LEFT JOIN`.

---

28. Find guests without bookings

Using a `LEFT JOIN`, display only guests who have never booked a room.

Expected columns:

```
guest_id
guest_name
city
```

Business Query Assignments

29. Total bookings per guest

Display:

```
guest_name
total_bookings
```

Include guests with zero bookings.

---

30. Total service spending per booking

Calculate:

```
SUM(quantity × service_price)
```

Display:

```
booking_id
guest_name
total_service_cost
```

Sort from highest service spending to lowest.

What this assessment covers

```
SELECT
WHERE
BETWEEN
IN
ORDER BY
COUNT
SUM
AVG
MIN
MAX
GROUP BY

INNER JOIN
LEFT JOIN
Multi-table JOIN
Calculated columns
Aggregation across JOINs
```